

# 김원혁

복잡한 데이터 특성을 파악해 모델 성능을 극대화하는, AI 분야 10년 차 AI/ML 엔지니어입니다 (석사 연구 포함). 보안 데이터와 같이 특수성이 강한 도메인에서도 최적의 모델을 설계하고 파인튜닝할 수 있는 응용력을 갖췄으며, TB급 대용량 데이터 처리부터 MLOps 파이프라인 구축까지 문제를 AI 기술로 해결하는 End-to-End 엔지니어링에 주력합니다. 최근에는 LLM 에이전트를 실제 서비스에 적용하는 영역으로 넓혔고, 특히 Claude Code를 적극 활용해 개발 하네스와 멀티에이전트로 개발 과정 자체를 자동화하며 여러 개인 프로젝트를 빠르게 구현하고 있습니다.

↗ GitHub :  
github.com/novatecoder

↗ 웹 포트폴리오 :  
https://novatecoder.github.io/portfolio/

## 학력

광운대학교 대학원 · 석사 로봇학과 (지능 시스템) 2017.03-2019.02

스케치 실사화를 위한 Multi Label Discriminator Generative Adversarial Nets

광운대학교 · 학사 로봇학부 지능시스템전공 2010.03-2017.02

## 핵심 역량

### ◆ 도메인 특화 LLM 고효율 학습·최적화 역량

DeepSpeed·Unsloth 기반 대규모 모델 파인튜닝 수행

LoRA·Sequence Parallelism·ZeRO Offloading 적용으로 학습 속도 50% 개선 및 동일 자원 대비 2배 규모 모델 학습 환경 구축

### ◆ 선례 없는 기술 과제의 서비스화 경험

가설 수립-실험-검증 기반 아키텍처 설계 (빠른 PoC → 확장형 설계 → 현업 적용)

보안 로그 등 특수 도메인 데이터 기반 커스텀 BERT 모델 설계부터 배포·운영까지 End-to-End 수행

### ◆ 협업 기반 모델 신뢰성 확보 및 활용도 제고

분석·제품팀과 협업하여 기술적 불확실성 해소

SNGP 적용을 통한 불확실성 지표 도입으로 모델 신뢰도 및 실무 활용도 향상

### ◆ End-to-End MLOps 자동화 및 데이터 신뢰성 확보

Airflow·Kubernetes 기반 학습·배포 파이프라인 구축

데이터 스키마 최적화로 조회 성능 70% 개선, 인프라 비용 30% 절감

## ◆ Agentic AI·LLM 애플리케이션 최신 기술의 빠른 실무 적용 역량

LangGraph·RAG·MCP·vLLM 기반 AI Agent를 실제 동작 프로토타입·서비스로 구현·검증 (서비스데스크 문의 자동 분류·RAG 자동응답 PoC → 사내 적용)

Claude Code 기반 개발 하네스 설계로 개발 과정 자체를 자동화(SDD·멀티에이전트 위임), 다수 개인 프로젝트 선구현

## 기술 스택

### LLM 앱·에이전트

LangGraph

RAG

MCP

Claude Code

멀티에이전트

SDD

vLLM

### LLM 파인튜닝·분산학습

LoRA

DeepSpeed

Unsloth

Sequence Parallelism

### Deep Learning & Modeling

TensorFlow

PyTorch

BERT

CNN

### Big Data & 분석

Scala

Spark

PySpark

Hadoop

Parquet

Python

Pandas

### MLOps & Infra

Kubernetes

Docker

Nvidia-docker

Airflow

MLflow

CI/CD

### 온디바이스·모바일

LiteRT-LM

Gemma

EXAONE

Kotlin

Android

### 데이터·클라우드

Faiss

pgvector

BGE-M3

FastAPI

MySQL

AWS(S3)

GCP

## 경력

2018.07-2025.08 (7년 2개월)

### 주식회사 안랩 (AhnLab)

AI 연구원 선임

#### Security LLM Fine-tuning 및 대규모 분산 학습 시스템 구축

2024.01-2025.08

**PROJECT GOAL** 악성 행위 탐지 및 보안 인텔리전스 분석을 위한 LLM 도입 및 효율적 파인튜닝 파이프라인 구축

##### 도메인 특화 성능 확보 (Domain Adaptation)

**PROBLEM** 범용 LLM의 보안 도메인 지식 부족으로 정밀한 악성 행위 탐지 불가

**ACTION** LoRA(Low-Rank Adaptation) 기법 도입, 보안 데이터셋으로 효율적 파인튜닝

**RESULT** 복잡한 악성 행위 패턴 탐지 성공, 적은 자원으로 보안 특화 모델 확보

##### GPU 메모리 병목 해결 및 아키텍처 최적화

**PROBLEM** 제한된 자원에서 대규모 모델 로딩 시 OOM 발생

**ACTION** DeepSpeed ZeRO Offloading-Unsloth로 메모리 최소화, Sequence Parallelism 도입

**RESULT** 동일 자원 내 기존 대비 2배 규모 모델 학습 환경 구현

##### 데이터 로딩 속도 및 배포 효율 개선

**PROBLEM** 대규모 텍스트 데이터 I/O 병목 및 수동 배포 비효율

**ACTION** PyTorch DataLoader 개선(Prefetching), Kubernetes 기반 CI/CD 파이프라인 구축

**RESULT** 학습 속도 50% 향상, 데이터 로딩 속도 2배 이상 향상

#### MLOps 기반 학습 파이프라인 자동화 및 성능 최적화

2023.01-2023.12

**PROJECT GOAL** 데이터 수집(ETL)부터 모델 학습까지 전 과정 자동화 및 데이터 신뢰성 확보

##### 학습 파이프라인 오케스트레이션

**PROBLEM** 데이터 처리~학습까지 수동 개입 빈번, 워크플로우 파편화

**ACTION** Apache Airflow 도입, '수집→전처리→학습→검증'을 DAG로 완전 자동화

**RESULT** 수동 개입 최소화로 모델 업데이트·배포 주기 획기적 단축

##### 데이터 스키마 재설계 및 품질 개선

**PROBLEM** 보안 데이터 불균형 및 비효율적 스키마로 학습 성능 저하

**ACTION** 병목 분석 후 데이터 스키마 재설계 및 전처리 로직 최적화

**RESULT** 데이터 신뢰도 확보 및 쿼리/로딩 속도 개선

## 보안 도메인 맞춤형 모델 설계/튜닝 및 기술 고도화

2021.01-2022.12

**PROJECT GOAL** 도메인 특화 이상 탐지 모델 설계 및 노후 프레임워크 전환(Migration)

### 보안 도메인 맞춤형 BERT 모델 개발

- PROBLEM** 일반 NLP 모델 구조로 보안 로그/악성코드 패턴 탐지 난해
- ACTION** BERT 모델 코드를 보안 로그 입력 구조에 맞게 Customizing
- RESULT** 범용 모델을 보안 도메인에 성공적으로 최적화

### 탐지율(Recall) 극복 및 신뢰성 강화

- PROBLEM** 모델의 과확신(Overconfidence) 및 낮은 재현율
- ACTION** 불확실성(Uncertainty) 지표 도입 및 화이트리스트 로직 앙상블
- RESULT** 오탐 최소화 및 Recall 집중 개선

### 기술 부채 해결 (Legacy Migration)

- PROBLEM** TensorFlow 1.x 노후화로 유지보수 제약
- ACTION** TensorFlow 2.x·PyTorch 2.x로 전체 코드 마이그레이션·리팩토링
- RESULT** 최신 모니터링 파이프라인 구축 및 확장성 확보

## 대용량 AI 정기 학습을 위한 전처리 파이프라인 구축

2019.05-2020.12

**PROJECT GOAL** TB급 보안 로그 데이터 저장 효율화 및 학습 전용 파이프라인 이중화

### 데이터 경량화 및 저장 효율화

- PROBLEM** TB급 로그 전수 저장으로 스토리지 비용 증가·쿼리 지연
- ACTION** Spark 기반 필터링 파이프라인, Parquet 포맷·파티셔닝 적용
- RESULT** 데이터 조회 속도 70% 단축, 클라우드 인프라 비용 30% 절감

### 학습 전용 데이터 파이프라인 최적화

- ACTION** 학습 최적화 포맷 TFRecord로 2차 가공하는 별도 파이프라인 구성
- RESULT** 학습 단계 전처리 부하 제거 및 모델 평가 속도 개선

#### 개발 환경 표준화 (Docker)

- PROBLEM** 폐쇄망 환경 내 연구원 간 환경 불일치로 협업 효율 저하
- ACTION** GPU 개발 이미지를 Docker로 표준화, Git 시스템 이관 주도
- RESULT** 연구/개발 협업 효율 극대화

#### 딥러닝 기반 악성코드 탐지 프로토타입

- ACTION** 바이너리 문자열을 이미지로 변환 후 CNN 모델 학습
- RESULT** 이미지 기반 딥러닝 탐지 기술 타당성 검증

## ● 개인 연구 및 LLM 심화 (2025.09-2026.03)

### 최신 AI 직접 구현 · 개발 하네스 설계

**PROJECT GOAL** 최신 AI(LLM·LangGraph·RAG·MCP)를 동작하는 시스템으로 직접 구현·검증

#### AI 개발 하네스 구축 (현재까지 지속)

- PROBLEM** Claude Code로 멀티프로젝트 운영 시 세션마다 맥락 유실·품질 편차·셀프리뷰 문제 발생
- ACTION** SDD 파이프라인·리드아키텍트 위임 사이클·환경적응 테스트 티어를 갖춘 개발 하네스 자체 설계
- RESULT** git clone 하나로 어느 머신에서든 동일 동작, 반복 개발 표준화 (현재까지 지속 개선)

#### 최신 LLM 패턴 선구현

- ACTION** LangGraph 멀티에이전트·자가교정 루프, 온프레미스 양자화 모델 RAG, MCP 표준 서버 등을 개인 프로젝트로 선구현
- RESULT** 가계부 AI·KIS MCP·RAG 문서QA 등 다수 공개 프로젝트 완성

## 메타넷디지털 (삼성 파견)

AI 엔지니어

### 서비스데스크 AI 자동화 (메인 담당)

**PROJECT GOAL** 서비스데스크 문의 자동 분류·라우팅 + RAG 자동응답 시스템 설계·구현 (메인 담당)

#### 문의 자동 분류·라우팅 + RAG 자동응답

**PROBLEM** 서비스데스크 문의를 담당자가 수동 분류·라우팅해 지연·부하 발생

**ACTION** LangGraph로 문의를 일반/전문지식/부서별 자동 분류, 단순 문의는 RAG로 직접 응답. 개인 PoC(ticket-router)로 E2E 검증 후 사내 적용

**RESULT** 2026.06 설계 착수, 실데이터 기반 구현 중 (서비스 오픈 전)

#### 사내 LLM 프로젝트 리드·레거시 정비

**ACTION** LangGraph 기반 사내 LLM 프로젝트 메인 담당(운영 안정화·아키텍처 개편), 레거시 코드베이스 리팩토링, 사내 문서 표준·오케스트레이션 도구 선정

**RESULT** 신규 환경 단기 안착

## 대표 프로젝트

### AI 개발 하네스

Claude Code로 여러 프로젝트를 운영하며 겪는 맥락 유실·품질 편차를 없애려 SDD 파이프라인·리드아키텍트 위임 사이클·환경적응 테스트 티어를 갖춘 개발 하네스를 직접 설계했습니다. 예제 하나를 기획부터 EARS 명세·에이전트 위임·검증까지 관통시킨 워크드 예제를 공개 쇼케이스 repo로 정리했습니다

Claude Code

SDD

서브에이전트

### 티켓 라우터

서비스데스크 문의를 담당 팀에 자동 분류·배정하고 근거가 충분한 단순 문의는 매뉴얼을 인용해 자동 응답하되, 민감하거나 근거가 부족한 문의는 안전 레이어(answerability)가 사람에게 이관합니다. 서비스데스크 대체 과제의 개인 PoC로 E2E까지 검증했습니다

LangGraph

RAG

FastAPI

PostgreSQL

vLLM

## sift

완전 오프라인 온디바이스  
LLM(LiteRT-LM+Gemma-4-E4B, GPU)이  
기기 안에서 문자를 직접 읽어 중요한 것만  
다이제스트로 선별하고 스미싱까지 잡아내, 문자  
원문이 기기 밖으로 나가지 않습니다  
(비공개·수익화 준비)

Kotlin

LiteRT-LM

Gemma-4-E4B

온디바이스 GPU

## 코딩테스트 퀴즈 앱

알고리즘 43유형·426문항을 7종 퀴즈와 간격  
반복(SRS)으로 자투리 시간에 복습하는 오프라인  
Android 앱으로, 단위 테스트 198건을 갖춰  
앱스토어 출시를 준비 중입니다 (비공개)

Kotlin

Room

SRS

### 이전 프로젝트

- ◆ **영화 분석 파인튜닝** — 영화 줄거리·메타데이터를 분석하도록 LLM을 파인튜닝한 초기 실험 프로젝트
- ◆ **가계부 AI 어드바이저** — 여러 에이전트가 스스로 쿼리를 교정하며 자연어 질문에 답하고, 가맹점 관계를 그래프로 분석해 이상 지출을 짚어주는 가계부 어드바이저 (SQL Injection 검증 노드 포함)
- ◆ **KIS MCP 서버** — 증권사(KIS) API를 MCP 표준 도구로 감싸 자연어 명령만으로 주식 잔고 조회·매매를 수행하는 중계 서버
- ◆ **Ollama RAG 문서QA** — 업로드 문서를 근거로만 답하는 RAG에 온프레미스 양자화 경량 모델을 붙여, 데이터 유출·환각 없이 문서에 질의하는 앱
- ◆ **투자 에이전트** — 뉴스·주가·재무 등 흩어진 정보를 통합해 LLM이 매수·매도 의견을 정리해 주는 투자 분석 보조 에이전트
- ◆ **PySpark NLP 파이프라인** — Data Skew를 Key Salt로 해소해 TB급 텍스트를 병렬 정제·토큰화하는 대용량 처리 파이프라인

### 진행 중

- ◆ **개인 AI 동반자** — 감정 엔진을 가진 캐릭터가 대화와 기억, 일정을 직접 관리하고 코딩 태스크는 하위 에이전트에 맡기는 오프라인 개인 AI 동반자 (초기 버전, 개선 중)
- ◆ **특허 출원 자동화** — 특허 명세서 초안과 출원 절차를 LLM으로 보조·자동화하는 도구 (출원 준비 중)
- ◆ **AI 투자 자문 시스템** — MCP로 시장 데이터를 통합해 투자 판단을 돕는 자문 시스템 (개발 중)

## 자기소개서

### — 지향점 및 포부 — 아이디어를 '동작하는 서비스'로 구현하는 실전형 AI 엔지니어

AI로 실생활과 산업 현장의 불편을 해소하는 데 흥미를 느껴 이 길을 걸어왔습니다. 지난 7년간 보안이라는 특수 도메인에서 '이론'을 '현장의 가치'로 전환하는 엔지니어링의 본질을 체감했고, 최근에는 생성형 AI·에이전트 기술을 실무에 적용하는 영역으로 역량을 넓혀왔습니다. 앞으로도 최신 AI 기술을 PoC로 빠르게 검증하고 확장형 시스템으로 안착시켜, 사용자가 체감할 수 있는 실질적 가치를 만드는 엔지니어가 되고자 합니다.

## — 직무 역량 — 모델 최적화부터 MLOps, 최신 에이전트까지 검증된 스택

안랩 7년 2개월간 AI 엔지니어링 전 과정을 End-to-End로 수행했습니다. Spark 기반 TB급 빅데이터 처리(조회 성능 70% 개선·인프라 비용 30% 절감), 보안 도메인 LLM 파인튜닝·최적화(LoRA·DeepSpeed·Unsloth, 학습 속도 50% 개선), Airflow·Kubernetes 기반 MLOps 자동화까지 담당했습니다. 이직 후에는 LangGraph 멀티에이전트·RAG·MCP 기반 AI Agent를 직접 구현·검증하고, Claude Code 기반 개발 하네스를 설계해 개발 과정 자체를 자동화하며 다수 개인 프로젝트를 선구현하고 있습니다.

## — 성장 과정 — '데이터를 더 넣을까'에서 '문제를 다시 정의할까'로

주니어 시절 보안 관제 모델의 오탐을 '틀린 데이터 재학습'으로만 대응했으나 해결되지 않았습니다. 수석 연구원의 '이 파일이 왜 탐지되면 안 되는지 분석했냐'는 질문을 계기로, 문제를 '모델 성능 개선'이 아닌 '파일 분석 기반 예외 처리(화이트리스트)'로 재정의해 해결했습니다. 이후 '어떻게 구현하지'보다 '왜 해야 하고 근본 원인이 무엇인지'를 먼저 묻는 공학적 사고를 체득했습니다.

## — 업무 방식 및 성격 — 궁즉변(窮則變)의 지혜와 PoC 기반의 기민함

'궁즉변 변즉통 통즉구'를 좌우명으로 삼아, 막힌 문제일수록 관점을 바꿔 돌파구를 찾습니다. 신중한 성향을 보완하기 위해 PoC 기반 단계별 일정 관리를 실천합니다 — 빠른 PoC로 타당성을 먼저 검증하고, 확장형으로 설계한 뒤, 기본 요구사항을 충족하고 신기능을 추가합니다. 흥미로운 기술은 TODO로 관리해 본류 작업의 리듬을 잃지 않습니다.

## — 문제 해결 능력 — 선례 없는 난제를 도메인 최적화로 돌파

레퍼런스가 전무한 상황에서 보안 로그라는 특수 데이터에 NLP를 접목했습니다. CNN·RNN·LSTM을 밑바닥부터 실험해 BERT 입력 구조 커스터마이징이 최선임을 증명했고, Airflow로 학습 파이프라인을 자동화하며 고객사 저사양 환경을 위해 경량화·양자화로 상용화까지 이끌었습니다. 다만 PoC부터 무작정 시작해 요구사항 변경으로 재작업한 경험에서, '무엇을 만들 것인가'의 초기 설계가 구현력만큼 중요함을 배웠습니다.